

DSAII- HW4

Tomas Salaz

March 9, 2026

Problem 1

Given the sequence we can assume the sizes of the matrices are as follows:

$$A_1 = 4 \times 2, A_2 = 2 \times 5, A_3 = 5 \times 1, A_4 = 1 \times 3.$$

For the recurrence, we let $M[i][j]$ be the minimum cost to multiply matrices i through j .

$M[i][j] = 0$ if there is one Matrix (no multiplication needed)

$M[i][j] = M[i][k] + M[k+1][j] + p[i-1] * p[k] * p[j]$ (min over all k from i to $j-1$, $p[i]$ is the sequence of the dimensions).

Using this we can start to make the tables. The 1st one is the M-Table We start with 0's on the diagonal, Then we look chains of length 2 (so $M[1][2], M[2][3], M[3][4]$).

$M[1][2]$ splits at $k = 1$ so $4 * 2 * 5 = 40$

$M[2][3]$ splits at $k = 2$ so $2 * 5 * 1 = 10$

$M[3][4]$ splits at $k = 3$ so $5 * 1 * 3 = 15$

so we get the diagonal above the 0's.

For chains of length 3, there are only 2 cases

CASE 1: $M[1][3]$ has splits at $k = 1, 2$, so we choose the min of the 2 cases

for $k = 1$ we have

$$M[1][1] + M[2][3] + p[0] * p[1] * p[3] = 0 + 10 + 4 * 2 * 1 = 18$$

for $k = 2$ we have

$$M[1][2] + M[3][3] + p[0] * p[2] * p[3] = 40 + 0 * 4 * 5 * 1 = 60$$

So we keep the min, 18. CASE 2: $M[2][4]$ has splits $k = 2, 3$ for $k = 2$

$$M[2][2] + M[3][4] + p[1] * p[2] * p[4] = 0 + 15 + 2 * 5 * 3 = 45$$

for $k = 3$

$$M[2][3] + M[4][4] + p[1] * p[3] * p[4] = 10 + 0 + 2 * 1 * 3 = 16$$

So we keep the min 16

	1	2	3	4
1	0	40	18	30
2		0	10	16
3			0	15
4				0

For the final chain there is only one case of length 4, the entire chain, with splits $k = 1, 2, 3$. The only case is

$$M[1][4]$$

for $k = 1$

$$M[1][1] + M[2][4] + p[0] * p[1] * p[4] = 0 + 16 + 4 * 2 * 3 = 40$$

for $k = 2$

$$M[1][2] + M[3][4] + p[0] * p[2] * p[4] = 40 + 15 + 4 * 5 * 3 = 115$$

for $k = 3$

$$M[1][3] + M[4][4] + p[0] * p[3] * p[4] = 18 + 0 + 4 * 1 * 3 = 30$$

So we keep 30 as the min.

The M-Table shows is that the total cost minimum is going to be 30, we need the S-Table, which can be derived using the k-split values from earlier when we were making the M-Table.

So for $s[1][2]$ there there was only one choice , $k = 1$

For $s[2][3]$, the only choice was $k = 2$, And so on

For splits with multiple k options, we take the k that gave the minimum

So for $s[1][3]$, the $k = 1$ gave the minimum cost so we we take the 1. This is the S-Table

	1	2	3	4
1	-	1	1	3
2		-	2	3
3			-	3
4				-

Finally we can use both table to get the optimal parentheses.

Starting from the top right and working towards the Diagonal, we get that the first split is

$$s[1][4] \rightarrow (A_1 A_2 A_3) A_4$$

The the next diagonal we choose the 1, so $s[1][3]$

$s[1][3] \rightarrow (A_1)(A_2 A_3)$ and the last diagonal before the 0's is irrelevant since we have a full parenthesization, that is

$(A_1(A_2 A_3))A_4$ which is verifiable by the costs of each multiplication.

$$A_2 A_3 = 2 * 5 * 1 = 10$$

$$A_1 = 4 * 2 * 1 = 8$$

$$A_4 = 4 * 1 * 3 = 12$$

$10 + 8 + 12 = 30$ thus this is the optimal split.

Problem 2

Part A

Based on the problem there the robber either Doesn't Rob (0) or Robs (1) the bank, a binary choice. There is a constraint that there must be a 10 day gap before the robber goes back to rob the bank.

The recurrence is

$r(i) = 0$ if $i \leq 0$

$r(i) = \max[r(i-1), b_i + r(i-10)]$

Note: $r(i-1)$ means the robber doesn't rob on day i . $b_i + r(i-10)$ means the robber robs on day i , while respecting the 10 day rule.

Part B

The program can be written using an array

```

r[0..n]    \\Create Array
r[0] = 0    \\Base Case

for i from 1 to n
    if i >= 10
        r[i] = max(r[i-1], b[i]+r[i-10])
    else
        r[i] = max(r[i-1], b[i])

return r[n]

```

The main loop iterates through the array once, doing one unit of work so the runtime is $O(n * 1) = O(n)$ runtime.

Problem 3

Part A

We can start by using the recurrence from Problem 2 and building upon this.

We need to add, 2 new constraints:

1. Let W_i be the amount of work day i costs.
2. Total work must be $< W$

Let $r(i, j)$ be the max revenue obtainable from day 1.. i using at most j work.

There are still 2 choices

1. Don't Rob on day i

This implies that no work is spent and the revenue is unchanged, thus we can represent this.

Best Revenue = $r(i-1, j)$

- 2 Rob on day i .

Robbing on day i costs W_i work and takes away from our budget j . So the 2nd constraint must hold, $j \geq W_i$ to afford the work.

b_i is still the revenue earned and we still must wait 10 days, so the previous revenue $r[i-10, j-W_i]$ remains the same.

The Best (prior) Revenue = $r(i-10, j-W_i)$.

So the full recurrence is

$$r(i, j) = \begin{cases} 0 & \text{if } i \leq 0 \text{ (No days left)} \\ r(i-1, j) & \text{if } j < W_i \text{ (Cant afford to rob)} \\ \max[r(i-1, j), b_i + r(i-10, j - W_i)] & \text{if } j \geq W_i \text{ (Max of Don't Rob or Rob)} \end{cases}$$

Part B

This program is more complex but is as follows

```

r[0..n][0..W] \\Create Table

\\Base Case (done with loop in practice)
r[i][j] = 0 for all i <= 0

\\Fill Table
for i from 1 to n:
    for j from 0 to W:
        if j < w[i]
            r[i][j] = r[i-1][j] \\Cant afford to rob
        else
            prev = r[max(0,i-10)][j-w[i]]
            r[i][j] = max (r[i-1][j], b[i] + prev)
return r[n][W] \\Top Right Entry in table

```

The Runtime of this algorithm is dominated by the 2 loops, One goes to n, the other to W. The work inside each inner loop is only 1 so the runtime is $O(n \cdot W)$

Problem 4

We have a chocolate bar with n chunks in a line, where each chunk has a value v_i . We must break the bar into k contiguous parts while maximizing the minimum value in each chunk.

We can use prefix sums, since we are left with the last chunk, we need a way to define the sum of the chunks before our piece.

Let $S[i] = v_1 + v_2 + \dots + v_i$, then the value of chunks $\ell..r = S[r] - S[\ell - 1]$

The subproblem needs to Capture the 'contiguous' chunks aspect, track the amount of used chunks, and build up the solution.

We can define $C[i][j] =$ the max possible min-value when splitting chunks $1..i$ into j parts.

Since we are concerned with our last chunk, we can define the start of the last part as $S[i] - S[m]$ implying that the last chunk begins at $m + 1$

This in turn means the first $j - 1$ parts cover $1..m$, thus $C[m][j - 1]$

The min value across all parts is given by $\min(C[m][j - 1], S[i] - S[m])$

From here we try all possible vals of M and take the Max value, hence Maxing the Min.

$$C[i][j] = \max \text{ over all } m \text{ from } j-1 \text{ to } i-1 \text{ of } \{\min(C[m][j-1], S[i] - S[m])\}$$

So all together

$$C(i, j) = \begin{cases} S[i] & \text{if } j = 1 \text{ (Prefix Sum)} \\ 0 & \text{if } j > i \text{ (Impossible Case)} \\ \max_{m=j-1}^{i-1} \{\min(C[m][j-1], S[i] - S[m])\} & \text{if } 2 \leq j \leq i \end{cases}$$

In Code it would look something like this

```
\\Compute Prefix Sum
S[0] = 0
For i from 1 to n: S[i] = S[i-1] + v[i]

\\Base Case
for i from 1 to n:
    C[i][1] = S[i]

\\Recur and Fill Table
for j from 2 to k:
    for i from j to n:
        C[i][j] = 0
        for m from j-1 to i-1:
            prev = min(C[m][j-1], S[i]-S[m])
            C[i][j] = max(C[i][j], prev)

return C[n][k]    \\Top Right Entry
```

The Runtime of the algorithm is dominated by the nested for loops, the other k and then inner n . Each cell of C requires iterating over M , which takes $O(n)$ time in the worst case.

Thus the total runtime is $O(n * n * k) = O(n^2k)$